

UNIVERSIDADE CATÓLICA DO SALVADOR  
BACHARELADO EM ENGENHARIA DE SOFTWARE



DOCUMENTAÇÃO TÉCNICA: COMPILADOR  
MINILANG

Amanda Moreira Rapold Souza  
Loren Vitoria Cavalcante Santos  
Tauan Gramacho dos Santos

# 1. Visão Geral do Projeto

O MiniLang Compiler é um compilador implementado em Python 3. Ele foi desenvolvido com o objetivo de demonstrar todas as etapas clássicas de um compilador moderno. Sua linguagem é simples e imperativa, com tipagem estática, suportando variáveis inteiras e booleanas, estruturas de controle de fluxo (if/else, while), operações aritméticas e relacionais, entrada e saída de dados.

## 1.1 Pipeline de Compilação

O código-fonte MiniLang passa pelas seguintes fases em sequência:

| Etapa                  | Módulo            | Descrição                                  |
|------------------------|-------------------|--------------------------------------------|
| 1. Análise Léxica      | lexer.py          | Transforma o texto em tokens               |
| 2. Análise Sintática   | parser.py         | Constrói a Árvore Sintática Abstrata (AST) |
| 3. Análise Semântica   | semantic.py       | Verifica tipos e declarações               |
| 4. Geração de IR       | ir_generator.py   | Produz código intermediário TAC            |
| 5. Geração de Bytecode | code_generator.py | Gera instruções para a VM                  |
| 6. Execução            | vm.py             | Máquina virtual baseada em pilha           |

## 1.2 Estrutura de Arquivos

| Arquivo           | Responsabilidade                                    |
|-------------------|-----------------------------------------------------|
| main.py           | Ponto de entrada da CLI; orquestra todo o pipeline  |
| ast_nodes.py      | Definição dos nós da Árvore Sintática Abstrata      |
| lexer.py          | Analizador léxico (tokenização)                     |
| parser.py         | Analizador sintático (parser descendente recursivo) |
| semantic.py       | Analizador semântico e tabela de símbolos           |
| ir_generator.py   | Gerador de código intermediário TAC                 |
| code_generator.py | Gerador de bytecode para a VM                       |
| vm.py             | Máquina virtual baseada em pilha                    |
| exemplos/         | Programas de exemplo válidos e com erros            |

## 2. A Linguagem MiniLang

MiniLang é uma linguagem imperativa de tipagem estática com escopo em blocos. Abaixo está a especificação completa de suas construções.

### 2.1 Tipos de Dados

- `int` - números inteiros (ex: 0, 42, -7)
- `bool` - valores booleanos: `true` ou `false`
- `string` - literais de texto entre aspas duplas, usados apenas em `print()`

### 2.2 Palavras Reservadas

| Palavra                  | Uso                                          |
|--------------------------|----------------------------------------------|
| <code>int, bool</code>   | Declaração de variáveis                      |
| <code>if, else</code>    | Condicional                                  |
| <code>while</code>       | Laco de repetição                            |
| <code>print</code>       | Saída de dados                               |
| <code>read</code>        | Entrada de dados (retorna <code>int</code> ) |
| <code>true, false</code> | Literais booleanos                           |

### 2.3 Operadores

- Aritméticos: `+`, `-`, `*`, `/` (apenas entre inteiros; divisão inteira)
- Relacionais: `<`, `>` (comparam inteiros, retornam `bool`)
- Igualdade: `==`, `!=` (operandos do mesmo tipo, retornam `bool`)
- Unário: `-` (negação de inteiro)

Precedência (do menor para o maior):

- `==` e `!=` (igualdade)
- `<` e `>` (comparação)
- `+` e `-` (soma e subtração)
- `*` e `/` (multiplicação e divisão)
- `-` unário (maior precedência)

### 2.4 Comentários

- Linha: `// comentário`
- Bloco: `/* comentário de múltiplas linhas */`

### 2.5 Gramática Formal (BNF simplificada)

```
program      -> statement* EOF
statement    -> varDecl | assignment | print | if | while
varDecl      -> ('int' | 'bool') IDENT ';'
assignment   -> IDENT '=' expression ';'
print        -> 'print' '(' expression ')' ';'
if           -> 'if' '(' expression ')' block ('else' block)?
```

```
while      -> 'while' '(' expression ')' block
block      -> '{' statement* '}'
expression -> equality
equality   -> comparison (('==' | '!=') comparison)*
comparison -> term (('<' | '>') term)*
term       -> factor (('+' | '-') factor)*
factor     -> unary (('*' | '/') unary)*
unary      -> '-' unary | primary
primary    -> NUMBER | true | false | STRING | read() | IDENT | '('
expression -> ')'
```

## 3. Módulos do Compilador

### 3.1 ast\_nodes.py — Nos da AST

Este módulo define a hierarquia de classes que representam cada construção da linguagem na forma de uma árvore. Todas as classes herdam de `ASTNode` e são dataclasses do Python, garantindo construção e comparação simplificadas.

| Classe     | Campos                               | Representa                          |
|------------|--------------------------------------|-------------------------------------|
| Program    | statements: List[ASTNode]            | Raiz da árvore; lista de comandos   |
| Block      | statements: List[ASTNode]            | Bloco delimitado por { }            |
| VarDecl    | var type, name                       | Declaracao de variavel              |
| Assignment | name, value: ASTNode                 | Atribuição de valor                 |
| Print      | value: ASTNode                       | Instrução de saída                  |
| Read       | (sem campos)                         | Leitura de entrada                  |
| If         | condition, then_branch, else_branch? | Condicional com else opcional       |
| While      | condition, body: Block               | Laco de repeticao                   |
| BinaryOp   | left, operator, right                | Operação binária                    |
| UnaryOp    | operator, operand                    | Operação unária (-)                 |
| Literal    | value, literal_type                  | Valor constante (int, bool, string) |
| Identifier | name: str                            | Referência a variável               |

### 3.2 lexer.py — Analisador Lexico

O Lexer é responsável pela primeira fase: ler o código-fonte caractere por caractere e produzir uma lista de tokens. Cada token carrega tipo, valor, linha e coluna, o que permite mensagens de erro precisas.

#### Classes

- **Token:** dataclass com campos `type`, `value`, `line`, `column`.
- **LexerError:** exceção lançada para caracteres inválidos ou strings/comentários não fechados.
- **Lexer:** classe principal que implementa a tokenização.

#### Tabelas de tokens

- `RESERVED` - palavras-chave mapeadas para seus tipos (ex: `'if' -> IF`)
- `SINGLE_CHAR_TOKENS` - símbolos de um caractere (`+`, `-`, `*`, `/`, etc.)
- `TWO_CHAR_TOKENS` - operadores de dois caracteres (`==`, `!=`)

#### Métodos principais

- **advance():** avança o cursor e atualiza linha/coluna.
- **peek():** inspeciona o caractere seguinte sem consumir.
- **skip\_whitespace\_and\_comments():** ignora espaços e comentários (`//` e `/* */`).

- **read\_identifier\_or\_reserved():** lê identificadores e diferencia palavras reservadas.
- **read\_number():** consome sequências de dígitos.
- **read\_string():** lê literais entre aspas duplas.
- **tokenize():** ponto de entrada; retorna List[Token] com EOF no final.

## Exemplo

Entrada:

```
int x;
```

Saída:

```
Token(INT, 'int', line=1, col=1)
Token(IDENT, 'x', line=1, col=5)
Token(SEMICOLON, ';', line=1, col=6)
Token(EOF, '', line=1, col=7)
```

## 3.3 parser.py — Analisador Sintático

O Parser recebe a lista de tokens do Lexer e constrói a AST usando o método de análise descendente recursivo (Recursive Descent Parser). Cada regra gramatical corresponde diretamente a um método da classe.

### Classe Parser

- **ParserError:** exceção lançada com mensagem detalhada indicando token inesperado, linha e coluna.
- **consume():** exige que o token atual seja do tipo esperado, avançando o cursor ou lançando erro.
- **match():** avança o cursor se o tipo atual estiver na lista esperada.

### Hierarquia de métodos de expressão (do menor para o maior)

- expression() -> equality()
- equality() -> comparison() com == e !=
- comparison() -> term() com < e >
- term() -> factor() com + e -
- factor() -> unary() com \* e /
- unary() -> primary() com - unário
- primary() -> literal, variável, read(), parênteses

Essa estrutura de chamadas encadeadas implementa a precedência de operadores de forma elegante: operadores com menor precedência ficam nas chamadas externas e os de maior precedência ficam nas mais internas.

## 3.4 semantic.py — Analisador Semântico

Após a construção da AST, o analisador semântico percorre a árvore verificando a correção lógica do programa: declaração prévia de variáveis, compatibilidade de tipos em atribuições e operações, e tipo correto nas condições de if e while.

### Tabela de Símbolos (SymbolTable)

Implementada como uma pilha de escopos (lista de dicionários). Ao entrar em um bloco, um novo escopo é empilhado; ao sair, é desempilhado. O lookup percorre a pilha de trás para frente, garantindo sombreamento de variáveis.

- **declare(name, type):** registra a variável no escopo atual ou lança erro de redeclaração.
- **lookup(name):** busca a variável nos escopos, do mais interno ao global.
- **snapshot():** retorna um dicionário plano com todos os símbolos visíveis.

### Regras de tipo

- Operadores aritméticos (+, -, \*, /): ambos os operandos devem ser int; resultado e int.
- Operadores relacionais (<, >): ambos devem ser int; resultado e bool.
- Igualdade (==, !=): ambos do mesmo tipo; resultado e bool.
- Condições de if e while devem ser bool.
- read() sempre retorna int.
- Atribuição: o tipo da expressão deve ser idêntico ao tipo declarado da variável.

### Erros detectados

- Variável usada sem ser declarada.
- Variável declarada mais de uma vez no mesmo escopo.
- Tipo incompatível em atribuição (ex: int x recebendo true).
- Condição de if ou while não booleana.
- Operadores aplicados a tipos incompatíveis.

### 3.5 ir\_generator.py — Geracao deCodigo Intermediario

O gerador de código intermediário traduz a AST para o formato TAC (Three-Address Code / Código de Três Endereços). Essa representação linear e independente de arquitetura é fácil de analisar e otimizar.

#### Formato TAC

Cada instrução tem no máximo dois operandos-fonte e um destino. Resultados intermediários são armazenados em temporários t1, t2, ... gerados automaticamente. Labels L1, L2, ... controlam o fluxo.

#### Exemplos de tradução

Atribuição com expressão:

```
x = 10 + 5 * 2;
```

Gera:

```
t1 = 5 * 2
t2 = 10 + t1
x = t2
```

Condicional if/else:

```
if (x > 0) { print(1); } else { print(0); }
```

Gera:

```
t1 = x > 0
ifFalse t1 goto L1
print 1
goto L2
L1:
print 0
L2:
```

Laço while:

```
while (x > 0) { x = x - 1; }
```

Gera:

```
L1:
t1 = x > 0
ifFalse t1 goto L2
t2 = x - 1
x = t2
goto L1
L2:
```

### 3.6 code\_generator.py — Geracao de Bytecode

O BytecodeGenerator traduz a mesma AST para um conjunto de instruções de pilha, destinadas a ser executadas pela máquina virtual. E um segundo backend independente do gerador de IR.

#### Instruções geradas



| Instrução            | Descrição                                           |
|----------------------|-----------------------------------------------------|
| DECLARE nome         | Registra variável no dicionário da VM com valor 0   |
| PUSH_INT n           | Empilha inteiro n                                   |
| PUSH_BOOL true false | Empilha booleano                                    |
| PUSH_STRING 'texto'  | Empilha string                                      |
| LOAD nome            | Empilha valor da variável                           |
| STORE nome           | Desempilha e armazena na variável                   |
| READ                 | Le inteiro da entrada e empilha                     |
| PRINT                | Desempilha e exibe na saída                         |
| ADD, SUB, MUL, DIV   | Operações aritméticas sobre os dois valores do topo |
| EQ, NEQ, LT, GT      | Comparações; empilham bool                          |
| NEG                  | Negação do topo (unário -)                          |
| LABEL nome           | Marcador de posição (nop em runtime)                |
| JUMP nome            | Salto incondicional para label                      |
| JUMP_IF_FALSE nome   | Desempilha; salta se falso                          |
| HALT                 | Encerra a Execução                                  |

### 3.7 vm.py — Máquina Virtual

A VirtualMachine interpreta diretamente o bytecode produzido pelo BytecodeGenerator. Ela é uma máquina de pilha classic (stack machine): todas as operações trabalham sobre uma pilha de valores.

#### Estruturas internas

- **stack:** pilha principal de Execução (lista Python).
- **variables:** dicionário de variáveis declaradas e seus valores.
- **labels:** dicionário pré-computado de label -> índice no bytecode, construído em `_index_labels()` antes da Execução.
- **ip:** instruction pointer; índice da instrução atual.
- **inputs:** lista de inteiros fornecidos via `--input`; se vazia, le do stdin.
- **output:** lista com todos os valores impressos (útil para testes).

#### Funcionamento

A VM percorre o bytecode instrução por instrução, interpretando cada opcode e atualizando a pilha, as variáveis e o ip. Saltos são implementados atribuindo diretamente o valor de ip a partir do dicionário de labels, sem qualquer busca linear.

## 4. Ponto de Entrada — main.py

O arquivo main.py implementa a interface de linha de comando do compilador usando argparse. Ele orquestra a chamada de todos os módulos em sequência e permite visualizar o resultado de cada etapa individualmente.

### 4.1 Função compile\_source

Executa todo o pipeline de compilação e retorna uma tupla com os cinco artefatos intermediários:

```
tokens, ast, symbols, ir, bytecode = compile_source(source)
```

### 4.2 Flags da CLI

| Flag             | Efeito                                      |
|------------------|---------------------------------------------|
| --tokens         | Imprime a lista de tokens gerada pelo Lexer |
| --ast            | Imprime a AST completa com pprint           |
| --symbols        | Imprime um snapshot da tabela de símbolos   |
| --ir             | Imprime o código intermediário TAC          |
| --bytecode       | Imprime o bytecode da VM                    |
| --run            | Executa o bytecode na máquina virtual       |
| --input N [N...] | Fornecer valores inteiros para read()       |

### 4.3 Tratamento de erros

Qualquer exceção das classes LexerError, ParserError, SemanticError, VMError ou FileNotFoundError é capturada e exibida com a mensagem ERRO: <descricao>, encerrando o processo com código 1.

## 5. Exemplos de Programas

### 5.1 valido1.mini — If/Else basico

```
// Programa válido simples
int x;
int y;
bool maior;

x = 10;
y = 5;
maior = x > y;

if (maior) {
    print(x);
} else {
    print(y);
}
```

Execução esperada: imprime 10 (pois  $x > y$  e verdadeiro).

Comando: `python main.py exemplos/valido1.mini --tokens --ast --symbols --ir --bytecode --run`

### 5.2 valido2\_while.mini — Laco While

```
// Programa válido com while
int x;
x = 5;

while (x > 0) {
    print(x);
    x = x - 1;
}
```

Execução esperada: imprime 5, 4, 3, 2, 1.

Comando: `python main.py exemplos/valido2_while.mini --run`

### 5.3 valido3\_read.mini — Entrada de Dados

```
// Programa com entrada
int numero;
numero = read();

if (numero > 10) {
    print(1);
} else {
    print(0);
}
```

Execução com input 8: imprime 0. Com input 15: imprime 1.

Comando: `python main.py exemplos/valido3_read.mini --run --input 8`

## 5.4 Exemplos de Erro

| Arquivo                          | Conteúdo         | Erro esperado                                 |
|----------------------------------|------------------|-----------------------------------------------|
| erro_variavel_nao_declarada.mini | x = 10;          | SemanticError: variável x não declarada       |
| erro_tipo.mini                   | int x; x = true; | SemanticError: tipo compatível                |
| erro_sintatico.mini              | int = x;         | ParserError: identificador esperado após tipo |

## 6. Fluxo Completo de Compilacao — Exemplo Detalhado

A seguir, o trecho 'x = 10 + 5;' é rastreado por todas as etapas do compilador.

### Etapa 1 — Tokens

```
Token(IDENT, 'x', line=1, col=1)
Token(ASSIGN, '=', line=1, col=3)
Token(NUMBER, '10', line=1, col=5)
Token(PLUS, '+', line=1, col=8)
Token(NUMBER, '5', line=1, col=10)
Token(SEMICOLON, ';', line=1, col=11)
```

### Etapa 2 — AST

```
Assignment(
  name='x',
  value=BinaryOp(
    left=Literal(10, 'int'),
    operator='+',
    right=Literal(5, 'int')
  )
)
```

### Etapa 3 — Análise Semântica

Verifica que x foi declarada como int e que BinaryOp(int + int) retorna int. Tipos compatíveis; nenhum erro.

### Etapa 4 — IR (TAC)

```
t1 = 10 + 5
x = t1
```

### Etapa 5 — Bytecode

```
PUSH_INT 10
PUSH_INT 5
ADD
STORE x
```

### Etapa 6 — Execução na VM

1. PUSH\_INT 10 -> stack: [10]
2. PUSH\_INT 5 -> stack: [10, 5]
3. ADD -> desempilha 5 e 10, empilha 15; stack: [15]
4. STORE x -> desempilha 15, variables['x'] = 15; stack: []

## **7. Decisões de Projeto**

### **7.1 Parser Descendente Recursivo**

A escolha do parser descendente recursivo (top-down) é didaticamente clara: cada regra gramatical vira um método Python, tornando o código muito legível e fácil de estender. A desvantagem é que gramáticas com recursão à esquerda exigem refatoração, o que não é o caso da MiniLang.

### **7.2 Dois backends: IR e Bytecode**

O projeto gera tanto código TAC (`ir_generator.py`) quanto bytecode para a VM (`code_generator.py`), ambos a partir da mesma AST. Isso ilustra que um compilador pode ter múltiplos backends para diferentes propósitos: otimização (IR) e Execução (bytecode).

### **7.3 Máquina Virtual em vez de Assembly nativo**

Em vez de gerar Assembly x86 ou ARM, o compilador produz bytecode para uma VM própria. Essa escolha mantém o foco nas etapas de compilação sem depender de conhecimentos de arquitetura de hardware. A VM é portátil e facilmente inspecionável.

### **7.4 Tabela de símbolos com pilha de escopos**

A tabela de símbolos usa uma lista de dicionários como pilha para implementar escopos aninhados. Isso segue o padrão clássico da maioria dos compiladores e demonstra o funcionamento do sombreamento de variáveis de forma natural.

### **7.5 `read()` sempre retorna int**

Para simplificar a análise semântica, o compilador define que `read()` retorna sempre int. Uma extensão natural seria suportar `read_bool()` ou tipar o `read` por contexto.

## 8. Possíveis Extensões

- Suporte a funções com parâmetros e retorno de valores.
- Tipo string declarável (atualmente strings só são suportadas como literais em print).
- Operadores lógicos && e || com curto-circuito.
- Arrays e indexação.
- Otimizações no IR: propagação de constantes, eliminação de código morto.
- Geração de Assembly MIPS ou x86-64 como backend alternativo.
- Sistema de tipos mais rico: float, char, enum.
- REPL interativo para explorar a linguagem.